

Divide and Conquer

Divide and conquer is a design principle: solve a problem by splitting it into smaller subproblems, solving them recursively, and combining their solutions.

Natural Explanation

Instead of attacking one large problem directly: - Break it into smaller pieces of the same type. - Solve each piece. - Merge the partial answers.

This is powerful when subproblems are much smaller and the combine step is efficient.

Formal Pattern

A typical divide-and-conquer algorithm has three phases: - Divide: split a size- n problem into a subproblems, each of size about n/b . - Conquer: solve subproblems recursively. - Combine: combine sub-results in $f(n)$ time.

This often yields a recurrence:

$$T(n) = aT(n/b) + f(n).$$

Why It Can Be Fast

If splitting reduces size aggressively (for example by half) and combine is near-linear, total work can become $O(n \log n)$ instead of $O(n^2)$.

Example (merge sort):

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

When It Works Well

- Subproblems are similar to the original problem.
 - Recursion depth is manageable.
 - Combine step is efficient.
-

Common Pitfalls

- Expensive combine step that destroys the benefit.
 - Large recursion overhead for tiny inputs.
 - Splits that are very unbalanced.
-

Related

- [[Merge Sort]]
- [[Binary Search]]
- [[Recurrence Relations]]
- [[Sorting Algorithms]]
- [[Lectures/Week 2]]